

Java Swing I

Grado en Ingeniería Informática
Universidad de Córdoba



Java Swing

1. Introducción
2. Ventanas
3. Componentes
4. Layout managers

Java Swing

1. Introducción

2. Ventanas

3. Componentes

4. Layout managers

Java Swing

Introducción

- *Swing* es una biblioteca de interfaces gráficas de usuario (GUI) para Java.
- Se incluye con el entorno de desarrollo de Java (JDK).
- Extiende la librería *awt*.
- Paquetes:
 - `javax.swing`
 - `java.awt`
 - `java.awt.event`

Java Swing

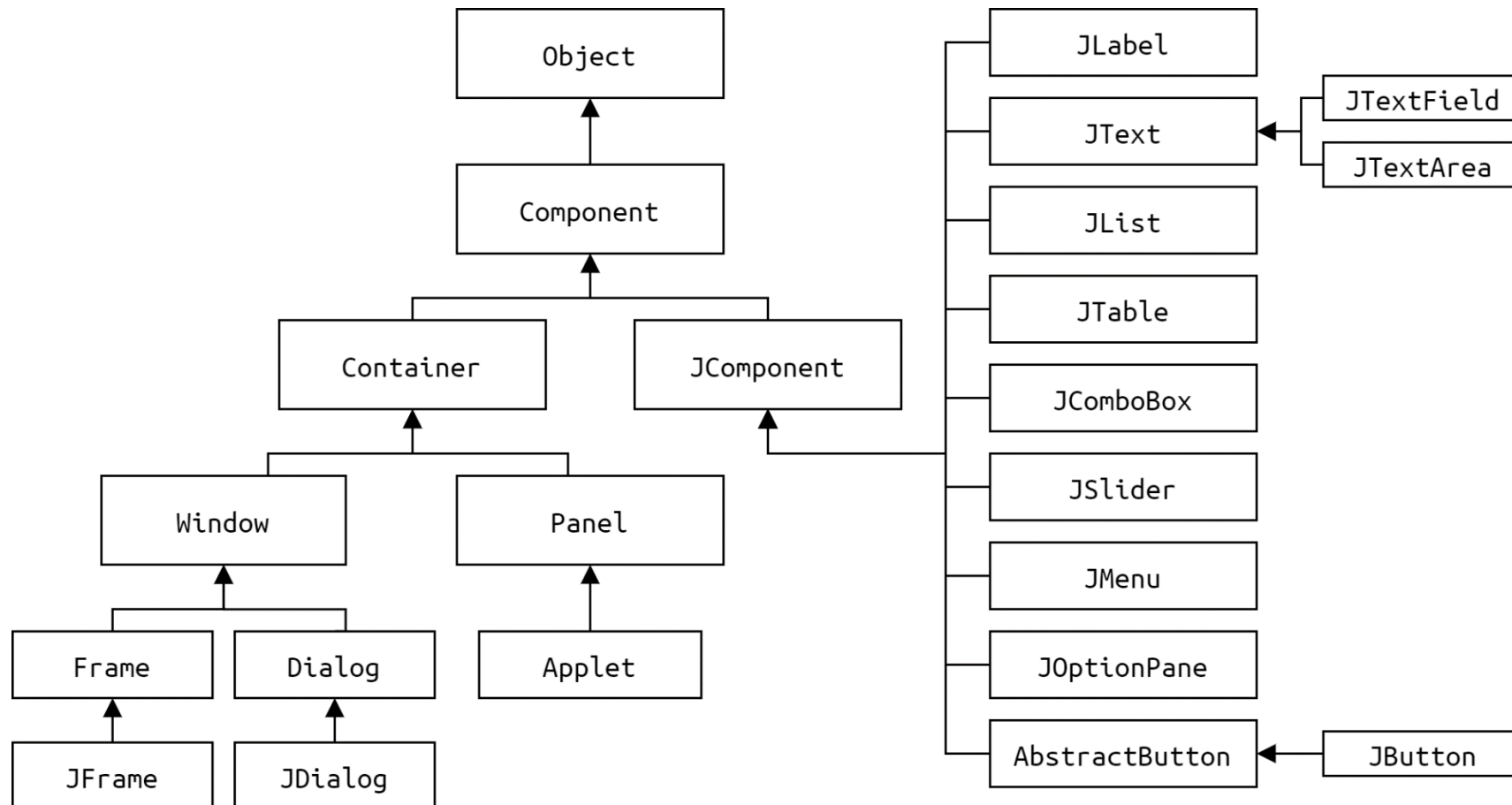
Introducción

▸ Java awt vs Java swing

Java awt	Java swing
Los componentes dependen de la plataforma	Los componentes son independientes de la plataforma
Componentes pesados	Componentes ligeros
No permite cambiar la interfaz en tiempo de ejecución	Permite modificar la interfaz en tiempo de ejecución (<i>pluggable look and feel</i>)
Menos componentes	Más componentes más potentes (tablas, listas, paneles con <i>scroll</i> , selectores de color, etc.)
No sigue modelo-vista-controlador	Sigue modelo-vista-controlador (MVC)

Java Swing

Introducción



Java Swing

Introducción

- Métodos comunes de los componentes (Component)
 - `void add(Component c)`
 - Añade un componente a otro componente
 - `void setSize(int width, int height)`
 - Define el tamaño del componente
 - `void setLayout(LayoutManager m)`
 - Indica qué estructura seguirá el componente
 - `void setVisible(boolean b)`
 - Indica la visibilidad del componente (por defecto, *false*)

Java Swing

1. Introducción
- 2. Ventanas**
3. Componentes
4. Layout managers

Java Swing

Ventanas

- La clase JFrame proporciona operaciones para manipular ventanas.

```
//Constructor
```

```
JFrame();
```

```
//Constructor con título ventana
```

```
JFrame(String titulo);
```

Java Swing

Ventanas

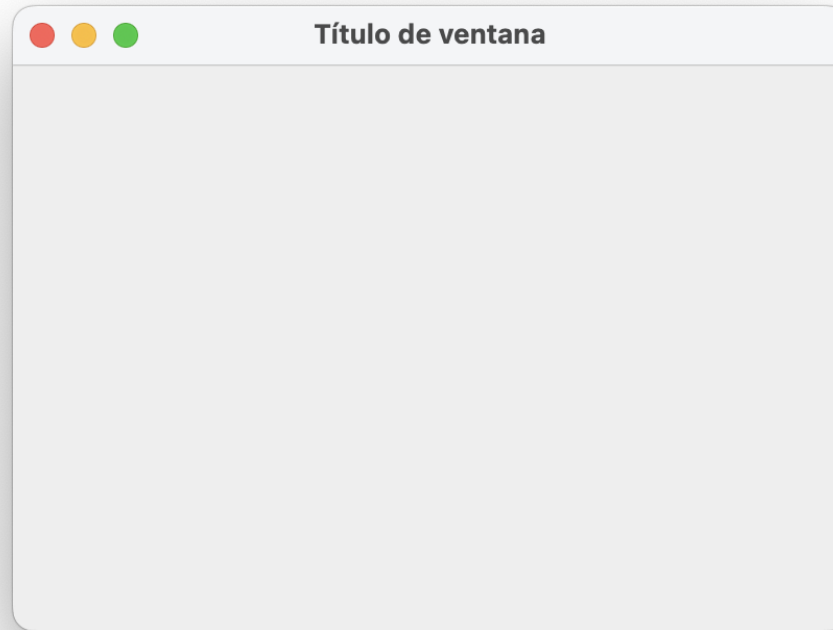
Example_GUI.java



```
public class Example_GUI {  
    public static void main(String[] args) {  
        JFrame jf = new JFrame("Título de ventana");  
  
        jf.setSize(400, 300);  
        jf.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        jf.setVisible(true);  
    }  
}
```

Java Swing

Ventanas



Java Swing

Ventanas

- Usualmente, se extiende la clase `JFrame`, para realizar las opciones de inicialización en el constructor.

```
public class MyFrame extends JFrame {  
    public MyFrame() {  
        super("Título de ventana");  
        setSize(400, 300);  
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    }  
}
```

Java Swing

1. Introducción
2. Ventanas
- 3. Componentes**
4. Layout managers

Java Swing

Componentes

- Dentro de una ventana se puede añadir distintos componentes

- JLabel

Etiqueta:

- JButton

Botón

- JTextField

Texto

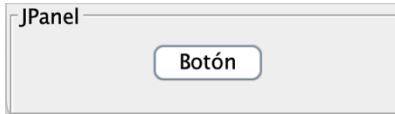
- JCheckBox

☐ Check box

- JRadioButton

☐ Radio button

- JPanel



- ...

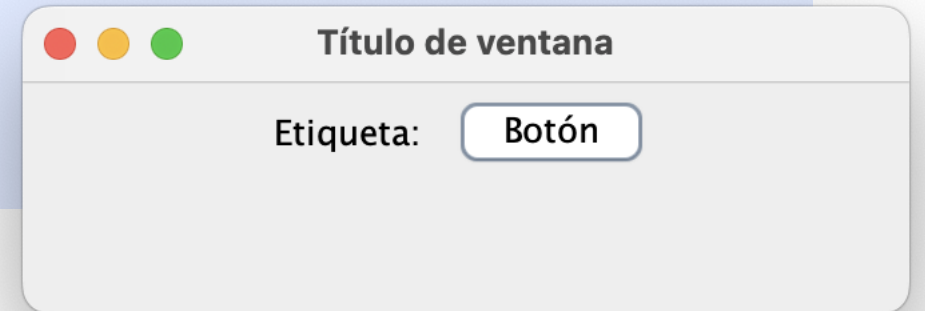
Java Swing

Componentes

```
Container cp = getContentPane(); //jf.getContentPane()

//Indicar estructura/layout
cp.setLayout(new FlowLayout());

//Añadir componentes
cp.add(new JLabel("Etiqueta: "));
cp.add(new JButton("Botón"));
```



Java Swing

Componentes

- JPanel es un componente que permite incluir dentro una estructura propia, permitiendo anidar componentes.

Example_JPanel.java



```
JPanel jp = new JPanel();  
jp.setLayout(new FlowLayout());  
jp.setBorder(new TitledBorder("Título JPanel"));  
jp.add(new JButton("Botón"));
```

Java Swing

1. Introducción
2. Ventanas
3. Componentes
- 4. Layout managers**

Java Swing

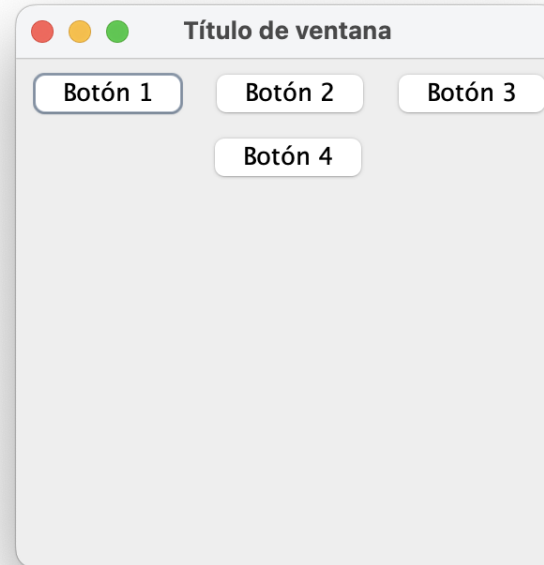
Layout managers

- No es habitual (ni práctico) indicar explícitamente la posición de los componentes dentro de la ventana.
- Los *layout managers* se encargan de colocar los componentes en la ventana, especificando la posición y tamaño de los mismos.
 - FlowLayout
 - GridLayout
 - BorderLayout
 - GridBagLayout
 - ...

Java Swing

Layout managers - *FlowLayout*

- FlowLayout
 - Coloca los elementos uno a continuación de otro.
 - Al redimensionar la ventana se pueden recolocar.



Java Swing

Layout managers - *FlowLayout*

Example_FlowLayout.java



```
FlowLayout f1 = new FlowLayout();  
  
//Alineación de los elementos  
f1.setAlignment(FlowLayout.CENTER);  
  
//Espaciado entre elementos  
f1.setHgap(5);  
f1.setVgap(5);
```

Java Swing

Layout managers - *GridLayout*

- **GridLayout**
 - Coloca los componentes en forma de tabla.
 - El orden en que se añadan determina su posición en la tabla.



Java Swing

Layout managers - *GridLayout*

Example_GridLayout.java



```
//Indicando filas y columnas
GridLayout gl = new GridLayout(3, 2);

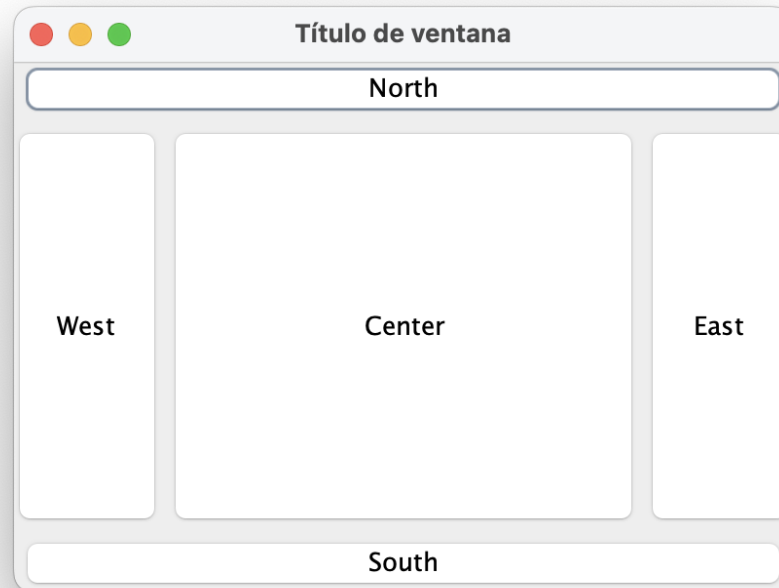
//Espaciado entre elementos
gl.setHgap(5);
gl.setVgap(5);

//Modificar número de filas/columnas a posteriori
gl.setRows(...);
gl.setColumns(...);
```

Java Swing

Layout managers - *BorderLayout*

- BorderLayout
 - Coloca los componentes para que se ajusten a los bordes y parte central de la pantalla.
 - Hay que especificar la colocación del elemento al añadirlo.



Java Swing

Layout managers - *BorderLayout*

Example_BorderLayout.java



```
BorderLayout b1 = new BorderLayout();

//Espaciado entre elementos
b1.setHgap(5);
b1.setVgap(5);

//Indicar colocación al añadir elementos
b1.getContentPane().add(new JButton(...),
                        BorderLayout.EAST);
```

Java Swing

Layout managers - *GridBagLayout*

- **GridBagLayout**
 - Más flexible (y complejo) que `GridLayout`.
 - Coloca los componentes en una malla de filas y columnas, permitiendo que algunos de ellos ocupen varias filas o columnas.
 - No todas las filas/columnas tienen que tener el mismo alto/ancho.
 - Cada elemento tiene asociado un objeto de la clase `GridBagConstraints`.

Java Swing

Layout managers - *GridBagLayout*

▸ GridBagLayout

```
JButton b = new JButton("Botón");  
GridBagConstraints gbc = new GridBagConstraints(...);  
  
getContentPane().add(b, gbc);
```

Java Swing

Layout managers - *GridBagLayout*

- **GridBagConstraints**

- **gridx, gridy**

- Columna/fila en la que se coloca (la esquina superior izquierda) el elemento. (Ambos empiezan por 0).

- Por defecto: **RELATIVE**.

- Se añade a continuación del último elemento añadido.

- **gridwidth, gridheight**

- Número de columnas/filas que ocupa el elemento.

- Se puede utilizar **REMAINDER** para que ocupe hasta la última columna/fila.

Java Swing

Layout managers - *GridBagLayout*

- *GridBagConstraints*

- *fill*

- Especifica qué hacer con el espacio sobrante en una celda.
 - NONE (*default*), HORIZONTAL, VERTICAL, BOTH.

- *ipadx, ipady*

- *Padding* interno horizontal/vertical a añadir a cada elemento.
 - El *padding* se añade a ambos lados del elemento.

- *insets*

- Especifica el espacio en blanco o *padding* externo a dejar en cada uno de los lados de la celda.
 - Utiliza un objeto de la clase *Insets*

Java Swing

Layout managers - *GridBagLayout*

- `GridBagConstraints`

- `anchor`

- Especifica el alineamiento de un componente en una celda.
 - `CENTER` (*default*), `NORTHWEST`, `NORTH`, `NORTHEAST`, `EAST`, `WEST`, `SOUTHWEST`, `SOUTH`, `SOUTHEAST`.
 - En versiones recientes se sustituyen por: `CENTER` (*default*), `FIRST_LINE_START`, `PAGE_START`, `FIRST_LINE_END`, `LINE_START`, `LINE_END`, `LAST_LINE_START`, `PAGE_END`, `LAST_LINE_END`.

<code>FIRST_LINE_START</code>	<code>PAGE_START</code>	<code>FIRST_LINE_END</code>
<code>LINE_START</code>	<code>CENTER</code>	<code>LINE_END</code>
<code>LAST_LINE_START</code>	<code>PAGE_END</code>	<code>LAST_LINE_END</code>

Java Swing

Layout managers - *GridBagLayout*

- `GridBagConstraints`

- `weightx`, `weighty`

- Permite indicar cómo se distribuye el espacio horizontal/vertical si la estructura resultante es menor que el área asignada.
 - Los valores van, por lo general, entre 0.0 (por defecto) y 1.0.
 - Mayores pesos indican que esa fila/columna necesitará un alto/ancho mayor en la estructura definida.
 - El alto/ancho de las filas/columnas se distribuye en base al mayor peso de alguno de los elementos en dicha fila/columna.

Java Swing

Layout managers - *GridBagLayout*

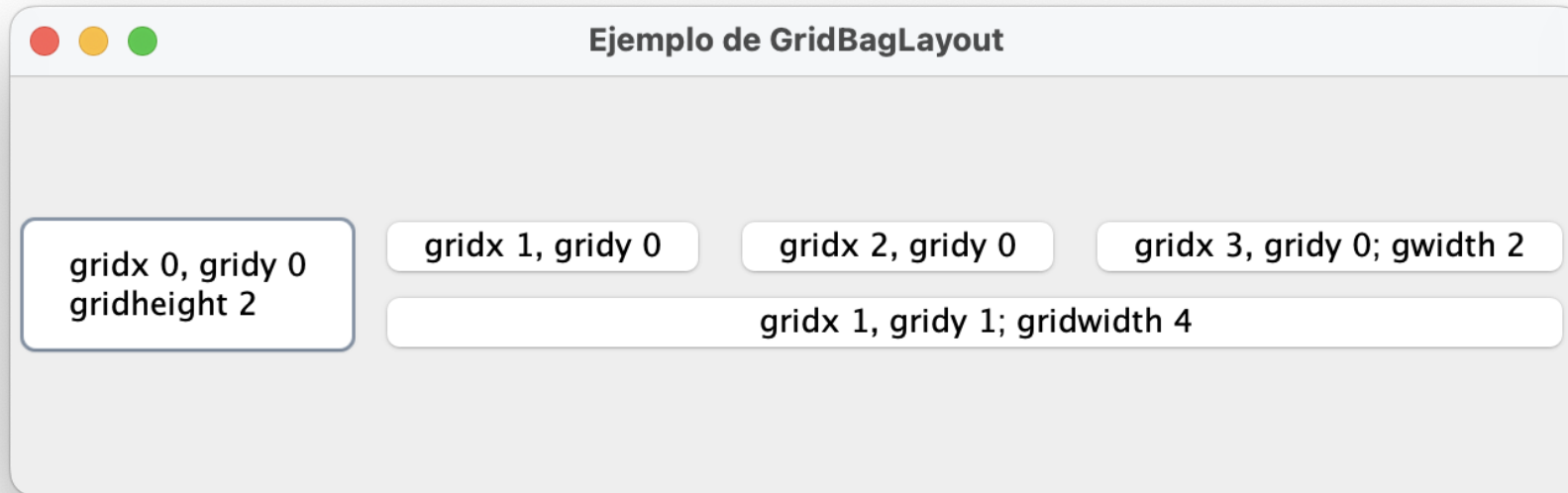
Example_GridBagLayout.java



```
JPanel panel = new JPanel(new GridBagLayout());  
frame.setContentPane(panel);  
GridBagConstraints c = new GridBagConstraints();  
  
c.fill = GridBagConstraints.BOTH;  
c.insets = new Insets(1, 2, 1, 2);  
c.gridx = 0; c.gridy = 0;  
c.gridheight = 2; c.gridwidth = 1;  
  
panel.add(new JButton("..."), c);
```

Java Swing

Layout managers - *GridBagLayout*



Java Swing

Layout managers

